

CT1 Exercises Data Transfer

1. What is a load/store architecture?

Data processing is between registers. Transfer of data from and to the external memory is done using load (memory to register) or store (register to memory) instructions.

2. What is the difference between a mov and a movs instruction?

mov	Transfer does NOT affect flags (low and high registers)
movs	Transfer affects flags (only low registers)

3. Which data transfer instructions should you use if at least one high register is an operand?

mov works for all registers (but only for reg/reg transfers)

4. List different ways of initializing a low register with an immediate value. What are the advantages/disadvantages?

mov <Rd>, #<imm8>

Value to load is in instruction

Less memory space but limited to 8-bit values.

ldr <Rt>, [pc, #<imm>]

Use pc/offset combination to point to the value to load.

Can be used to load larger values (up to 32-bit). Takes more space in memory.

5. What is a pseudo-Instruction? Explain what is done with a <ldr Rn, =literal> pseudo-instruction.

A pseudo-Instruction does not directly translate in machine code. It is an instruction that is interpreted (or expanded) by the assembler (the tool) to generate the needed machine code instruction(s).

When we run the assembler (the tool), the <ldr Rn, =literal> instruction is decomposed into a place reservation of the type .word and an indirect load of the type **ldr <Rt>, [pc, #<imm>]** to get the contents of the reserved position and write it in Rn

Literal can be an address (a position in the memory)

Literal can be a constant (the value is known at assembling time)

6. Write down the assembly instructions to perform the following actions.
7. Copy contents of r1 in r3 (flags unchanged)

```
mov r3,r1
```

8. Initialize r0 with 0xAA (flags unchanged)

```
ldr r0,=0xAA
```

000000AA will be placed in literal pool and loaded in r0 when the instruction executes
You cannot use a movs here, because it modifies flags.
To load an immediate value in the register of a cortex M0 without modifying the flags, you can use a ldr instruction.

9. Initialize r1 with 234 (flags modified)

```
movs r1,#234
```

10. Initialize r4 with 0x55AACC

```
ldr r4,=0x55AACC
```

11. Copy contents of r9 in r3.

```
mov r3,r9
```

12. Initialize r10 with 0x345678

```
ldr r0,=0x345678 @r0 as example. Another low register is ok  
mov r10,r0
```

1. Copy contents of r8 in r9

```
mov r9,r8
```

13. Whenever possible, work out the contents of registers or memory positions that have changed.

again:	ldr	r0,=0xFF	r0 = 0x000000FF
	ldr	r1,lit1	r1 = 0x0000EFAA
	ldr	r2,lit2	r2 = 0x00012345
	ldr	r3,=0x55AA	r3 = 0x000055AA
	mov	r4,r1	r4 = 0x0000EFAA
	mov	r4,r2	r4 = 0x00012345
	mov	r6,r3	r6 = 0x000055AA
	movs	r7, #04	r7 = 0x00000004
	ldr	r0,=lit1	r0 = address of lit1 (Address is not known)
	ldr	r1,=lit2	r1 = address of lit2
	ldr	r2,=lit3	r2 = address of lit3
	ldrb	r5,[r2]	r5 = 0x00000097
	ldrh	r6,[r2,#2]	r6 = 0x0000008F
	ldr	r2,[r0]	r2 = 0x0000EFAA
	ldr	r3,[r0,#4]	r3 = 0x00012345
	str	r3,[r1]	lit2 = 0x00012345
	str	r2,[r1,#8]	lit4 = 0x0000EFAA
	str	r4,[r1,r7]	lit3 = 0x00012345 (writes to (address of lit2 + r7))
	ldr	r5,=lit5	r5 = address of lit5
	movs	r0,#0	r0 = 0x00000000
	adds	r7,r0,#1	r7 = 0x00000001
	ldrsb	r6,[r5,r0]	r6 = 0xFFFFFFFF89
	ldrsh	r6,[r5,r7]	Problem: Unaligned memory access
	b	Again	
lit1:	.word	0xEFAA	
lit2:	.word	0x12345	
lit3:	.word	0x8F1097	
lit4:	.word	0xFF76552F	
lit5:	.word	0xAA654389	
lit6:	.word	0x0165	
Var1:	.word	0x23	
Var2:	.word	0x24	
Var3:	.word	0x23455678	
Var4:	.word	0xE4568900	

14. Consider the following assembled listing. Use table below for results of a) and b).

- For all the `ldr` instructions, calculate the value of `imm` and write the instructions in the form `ldr <Rt>, [pc, #<imm>]` (give `imm` in hex or decimal, as you want)
- Replace `yyyy` with the appropriate machine code
- Make sure you understand what the `<b loop>` instruction does.
- What do you think will happen during the program execution if there was no `<b loop>` instruction after `add r0, r1`? (Check the slides if you do not know.)

```

                                .equ CONST_VALUE_X, 123456
00000000
00000000          loop:
00000000      yyyy          ldr    r0, =0xFF55AAB0
00000002      yyyy          ldr    r1, =CONST_VALUE_X
00000004      yyyy          ldr    r2, myval2
00000006      yyyy          ldr    r3, myval3
00000008      yyyy          ldr    r4, myval4
0000000A      yyyy          ldr    r5, myval5
0000000C      yyyy          ldr    r6, myval6
0000000E      yyyy          ldr    r7, myval7
00000010      4408          add    r0, r1
00000012      E7F5          b      loop
00000014      33445566      myval3: .word    0x33445566
00000018      0000FFAA      myval4: .word    0x00FFAA
0000001C      00110044      myval5: .word    0x110044
00000020      00AABBCC      myval7: .word    0xAABBCC
                                @ Beware the order of variables
00000024      7788ABCD      myval6: .word    0x7788ABCD
00000028      33445555      myval2: .word    0x33445555
0000002C      FF55AAB0
                                00123456

```

`imm` is shown in decimal and hex to allow you to see both.

Code (yyyy)	imm in hexadecimal	imm in decimal
480A	ldr r0, [pc, #0x28] @	ldr r0, [pc, #40]
Example		
490B	ldr r1, [pc, #0x2C] @	ldr r1, [pc, #44]
4A08	ldr r2, [pc, #0x20] @	ldr r2, [pc, #32]
4B03	ldr r3, [pc, #0x0C] @	ldr r3, [pc, #12]
4C03	ldr r4, [pc, #0x0C] @	ldr r4, [pc, #12]
4D04	ldr r5, [pc, #0x10] @	ldr r5, [pc, #16]
4E05	ldr r6, [pc, #0x14] @	ldr r6, [pc, #20]
4F04	ldr r7, [pc, #0x10] @	ldr r7, [pc, #16]

Note: Remember that the pc contains the address of the next instruction. If the address is not word-aligned, the next upper word-address will be used for calculation.

`<b loop>` branches unconditionally to `loop`. Without this, the CPU will continue to fetch and execute and take the following contents of memory as code and try to interpret and execute it. Dangerous!

15. In the following C program, code the statements in **bold green** in assembly.

```
// C-Code
uint32_t x;
uint32_t y;
uint32_t *xp;

void main(void) {
    x = 3;
    xp = &x;
    y = *xp;
}
```



```
AREA myVar, DATA, READWRITE

@ Definition of variables in RAM space.
X      .word      0
Y      .word      0
XP     .word      0

AREA myCode, CODE, READONLY

main:
    @ This program is written in order to make it easy to understand
    @ Efficiency is not the important aspect in this implementation

    @ First point to variables using registers

    ldr r3,=X @ initialize r3 with address of X
                @ reserve place to store address of X in literal pool

    ldr r4,=Y @ initialize r4 with address of Y
                @ reserve place to store address of Y in literal pool

    ldr r5,=XP @ initialize r5 with address of XP
                @ reserve place to store address of XP in literal pool

    @ X = 3
    ldr r0,=0x03 @ load '3'
    str r0,[r3]  @ and store it in X

    @XP = &X
    str r3,[r5]  @ store address of X (that address is in r3)
                @ in XP (address of XP is in r5)

    @Y = *XP
    ldr R2,[r5]  @ read contents of XP variable (pointed to by
                @ r5) into R2. Address of X now in R2
    ldr R1,[R2]  @ read contents of X in R1
    str R1,[r4]  @ store R1 (contents of X) in Y (pointed to by r4)
```

16. In the following C program, code the statements in **bold green** in assembly. Use the frame for solution on following page.

```
// C-Code
uint8_t demoArray[2];
uint8_t *xp;

void main(void) {
    demoArray[0] = 10;

    demoArray[1] = 11;

    xp = demoArray;

    *xp = 111;

    xp++;

    *xp = 112;
}
```

Note: Other statements of the C program are shown in frame with machine code and addresses as examples to help you.

You are only required to write the assembly code (not the machine code).

Frame for solution

Possible solution, as given by the C compiler. This solution does not use pseudo instructions. Before looking at it, make sure that you understand the solution of 9a

```

22:      demoArray[0] = 10;
0x08000254 200A      movs      r0,#0x0A
0x08000256 4909      ldr       r1,[pc,#36] ; @0x0800027C
0x08000258 7008      strb      r0,[r1,#0x00]

23:      demoArray[1] = 11;
0x0800025A 200B      movs      r0,#0x0B
0x0800025C 7048      strb      r0,[r1,#0x01]

24:      xp = demoArray;
0x0800025E 4608      mov       r0,r1
0x08000260 4907      ldr       r1,[pc,#28] ; @0x08000280
0x08000262 6008      str       r0,[r1,#0x00]
25:      *xp = 111;
0x08000264 206F      movs      r0,#0x6F
0x08000266 6809      ldr       r1,[r1,#0x00]
0x08000268 7008      strb      r0,[r1,#0x00]

26:      xp++;
0x0800026A 4805      ldr       r0,[pc,#20] ; @0x08000280
0x0800026C 6800      ldr       r0,[r0,#0x00]
0x0800026E 1C40      ADDS      r0,r0,#1
0x08000270 4903      ldr       r1,[pc,#12] ; @0x08000280
0x08000272 6008      str       r0,[r1,#0x00]

27:      *xp = 112;
0x08000274 2070      movs      r0,#0x70
0x08000276 6809      ldr       r1,[r1,#0x00]
0x08000278 7008      strb      r0,[r1,#0x00]

28: }
0x0800027A 4770      bx        lr ;Not part of solution
0x0800027C 0000      .hword    0x0000
0x0800027E 2000      .hword    0x2000
0x08000280 0004      .hword    0x0004
0x08000282 2000      .hword    0x2000

```

This correction shows opcode and assembly instruction to help you understand more. For the solution, only the assembly is required.

17. Consider the following listing.

```

08000008 4804      again  ldr    r0, lit1
0800000A 480A      ldr    r0, =lit1
0800000C 4A04      ldr    r2, lit2
0800000E 4A0A      ldr    r2, =lit2
08000010 4B04      ldr    r3, lit3
08000012 4C04      ldr    r4, lit3
08000014 4E09      ldr    r6, =lit4
08000016 4F09      ldr    r7, =lit4
08000018 4408      add    r0, r1
0800001A E7F5      b      again

0800001C 00000001  xxx    .word    0x01 @ xxx stands for a label
08000020 00000002  xxx    .word    0x02
08000024 00000003  xxx    .word    0x03
08000028 00000004  xxx    .word    0x04
0800002C 00000005  xxx    .word    0x05
08000030 00000006  xxx    .word    0x06
08000034
          00000000
          00000000
          00000000
          00000000

```

- a) Compute on the basis of the opcode (marked in blue) the position where data is read to load registers (see table with example)

Instruction address	Opcode (marked in blue)	position where data is read	Contents of affected register after operation
08000008	4804	0x0800001C	r0 = 0x00000001

- b) The opcodes for <ldr r0, lit1> and <ldr r0, =lit1> are the same. But where are the operands? Explain the differences between the 2.
- c) Compute the contents of the registers after each instruction is executed once. Write results in table in a).

Example solutions will be written here

Instruction address	Opcode	position where data is read	Contents of affected register after operation
08000008	4804	0x0800001C	r0 = 0x00000001
0800000A	480A	0x08000034	r0 = 0x0800001C
0800000C	4A04	0x08000020	r2 = 0x00000002
0800000E	4A0A	0x08000038	r2 = 0x08000020
08000010	4B04	0x08000024	r3 = 0x00000003
08000012	4C04	0x08000024	r4 = 0x00000003
08000014	4E09	0x0800003C	r6 = 0x08000028
08000016	4F09	0x0800003C	r7 = 0x08000028

b) <ldr r0, lit1> The value at position label lit1 is loaded in r0

<ldr r0, =lit1> Of the form ldr r0, =label
(Room is made in the literal pool to store the address of lit1. That address is then loaded in r0 when code is executed)

<ldr r0, =const> (e.g. in previous exercise ldr r0, =0xFF55AAB0)
Of the form ldr r0, = constant (The constant is placed in a literal pool and loaded in r0)

c) see table in a)

Some help to understanding:

The code in the processor's memory with comments is shown below

```

0x08000008 4804      ldr      r0,[pc,#16]  @ 0x0800001C
0x0800000A 480A      ldr      r0,[pc,#40]  @ 0x08000034
0x0800000C 4A04      ldr      r2,[pc,#16]  @ 0x08000020
0x0800000E 4A0A      ldr      r2,[pc,#40]  @ 0x08000038
0x08000010 4B04      ldr      r3,[pc,#16]  @ 0x08000024
0x08000012 4C04      ldr      r4,[pc,#16]  @ 0x08000024
0x08000014 4E09      ldr      r6,[pc,#36]  @ 0x0800003C
0x08000016 4F09      ldr      r7,[pc,#36]  @ 0x0800003C
0x08000018 4408      add      r0,r0,r1
0x0800001A E7F5      b        Again (0x08000008)
0x0800001C 0001      .hword    0x0001      @ This is lit1.Value
0x00000001
0x0800001E 0000      .hword    0x0000
0x08000020 0002      .hword    0x0002      @ This is lit2.Value
0x00000002
0x08000022 0000      .hword    0x0000
0x08000024 0003      .hword    0x0003      @ This is lit3
0x08000026 0000      .hword    0x0000
0x08000028 0004      .hword    0x0004
0x0800002A 0000      .hword    0x0000
0x0800002C 0005      .hword    0x0005
0x0800002E 0000      .hword    0x0000
0x08000030 0006      .hword    0x0006
0x08000032 0000      .hword    0x0000
0x08000034 001C      .hword    0x001C      @ Address of lit1 stored
here
0x08000036 0800      .hword    0x0800
0x08000038 0020      .hword    0x0020      @ Address of lit2 stored
here
0x0800003A 0800      .hword    0x0800

```

0x0800003C	0028	.hword	0x0028
0x0800003E	0800	.hword	0x0800
0x08000040	0000	.hword	0x0000
0x08000042	0000	.hword	0x0000

Details to the solution for 9a, (given to help you see what is happening).

19:		ldr r3,=X	0x080007B8 4B05	
ldr r3,[pc,#20]	; @0x080007D0			
20:		ldr r4,=Y	0x080007BA 4C06	
ldr r4,[pc,#24]	; @0x080007D4			
21:		ldr r5,=XP	0x080007BC 4D06	ldr
r5,[pc,#24]	; @0x080007D8			
22:				
23:		; X = 3		
24:		ldr r0,=0x03	0x080007BE 4807	ldr
r0,[pc,#28]	; @0x080007DC			
25: loop1:		str r0,[r3]	0x080007C0 6018	str
r0,[r3,#0x00]				
26:				
27:		;XP = &X		
28:		str r3,[r5]	0x080007C2 602B	str
r3,[r5,#0x00]				
29:				
30:		;Y = *XP		
31:		ldr r2,[r5]	0x080007C4 682A	ldr
r2,[r5,#0x00]				
32:		ldr r1,[r2]	0x080007C6 6811	ldr
r1,[r2,#0x00]				
33:		str r1,[r4]	0x080007C8 6021	str
r1,[r4,#0x00]				
34:				
35:		; this not asked for in the exercise		
36:		adds r0, #01	0x080007CA 3001	adds
r0,r0,#0x01				
37:		b loop1	0x080007CC E7F8	
b	0x080007C0			
0x080007CE 0000	.hword	0x0000		

Another possible solution for exercise 9 a.

As given by the C compiler when code was written in C.

Note the movs. (This is not always good because it affects flags. So it should be used carefully)

This solution does not use pseudo instructions
 It is here for you to see what the compiler could do
 In a normal solution in assembler, it is better to use pseudo instructions in order to create the literal pool. See solution above.
 The lines in blue are only there for help.

```

                                @ x = 3;
0x08000254 2003                movs    r0,#0x03
0x08000256 4905                ldr     r1,[pc,#20] ; @0x0800026C
0x08000258 6008                str     r0,[r1,#0x00]
                                @ xp = &x;
0x0800025A 4608                mov     r0,r1
0x0800025C 4904                ldr     r1,[pc,#16] ; @0x08000270
0x0800025E 6008                str     r0,[r1,#0x00]
                                @ y = *xp;
0x08000260 4608                mov     r0,r1
0x08000262 6800                ldr     r0,[r0,#0x00]
0x08000264 6800                ldr     r0,[r0,#0x00]
0x08000266 4903                ldr     r1,[pc,#12] ; @0x08000274

```